DATABASE CONNECTIVITY

Visual Basic 6.0 | OAVS Computer Teacher Exam - Focused Study Notes

Scope: a. How VB access data | b. Introduction to OLE DB & ADO (What is OLE DB, OLE DB and ADO, SQL Statements, ADO Data Control, Coding ADO, Data Environment, Crystal Report). Colour key: **yellow = exam points**, **red = common mistakes**.

a. How VB access data

i. Data access interface

Definition: A data access interface is a set of objects/functions (an API) that lets a VB program connect to a database, send commands and receive results *without knowing the internal file format* of that database. **Purpose:** give VB a standard, database-independent way to read and update data.

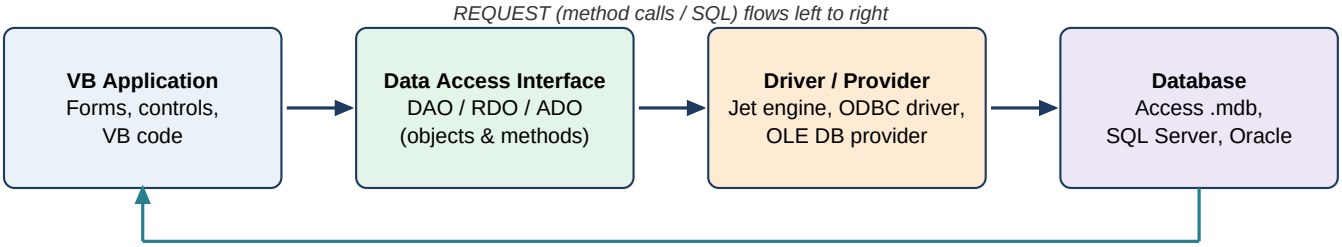

Fig 1: VB - Data Access Interface - Database flow

Data access interfaces used with VB

Interface	Full form	Works through	Used for
ODBC	Open Database Connectivity	ODBC drivers + DSN	Standard C-style API for SQL databases
DAO	Data Access Objects	Jet engine	Local Access (.mdb) databases; intrinsic **Data control**
RDO	Remote Data Objects	ODBC	Client/server DBs (SQL Server, Oracle); **RemoteData control**
ADO	ActiveX Data Objects	OLE DB	Latest and recommended in VB 6.0; **ADO Data Control**

Two ways VB works with data

- **Data-bound (design-time):** a data control (ADODC) + bound controls; very little code.
- **Code-based (run-time):** create ADO objects (Connection, Recordset) in code; full control.

OAVS EXAM POINTS

- Evolution: ODBC → DAO → RDO → ADO (ADO is newest, built on OLE DB).
- DAO = Jet; RDO = ODBC; ADO = OLE DB.
- Intrinsic Data control uses DAO; ADODC uses ADO.

COMMON MISTAKES / CONFUSING POINTS

- ADO/DAO are **interfaces**, not databases.
- Confusing the intrinsic **Data** control (DAO) with the **ADO Data Control**.

Summary: VB never talks to a database file directly; it uses an interface (DAO, RDO or ADO) and a driver/provider. ADO is the standard in VB 6.0.

ii. Relational database concepts

Definition: A relational database stores data in **tables** (relations) made of rows and columns; tables are linked through common fields. VB reads such tables into memory as a **Recordset**.

Term	Meaning	What it means in VB / ADO
Table (Relation)	Collection of related data in rows and columns	RecordSource / table name
Record (Row, Tuple)	One complete entry in a table	Current record of a Recordset
Field (Column, Attribute)	One data item of a record	Fields("Name") / `rs!Name`
Primary key	Field(s) that uniquely identify each record; no duplicates, no Null	e.g. RollNo, ClassID
Foreign key	Field that matches the primary key of another table	Links two tables (ClassID in Student)
Relationship	One-to-one, one-to-many, many-to-many between tables	Used in joins / Data Environment child commands

Term	Meaning	What it means in VB / ADO
Recordset	Set of records returned by a table or query held in memory	`ADODB.Recordset`

CLASS table (parent)

ClassID (PK)	ClassName
C1	Class X
C2	Class XI

1 ------ M
one class has many students

STUDENT table (child)

RollNo (PK)	Name	ClassID (FK)
1	Ravi	C1
2	Asha	C2
3	Kiran	C1

Fig 2: Relational database - tables, records (rows), fields (columns), primary key (PK), foreign key (FK) and a one-to-many relationship

OAVS EXAM POINTS

- A primary key must be **unique and not Null**.
- Foreign key value must exist in the parent table's primary key.
- Row = record = tuple; Column = field = attribute.

COMMON MISTAKES / CONFUSING POINTS

- Primary key vs foreign key: PK identifies a record in *its own* table; FK refers to *another* table.
- A Recordset is a *copy in memory*, not the table itself.

Summary: Relational data = tables (records and fields) linked by keys. VB brings a table/query result into a Recordset.

b. Introduction to OLE DB & ADO

i. What is OLE DB

Definition: OLE DB (*Object Linking and Embedding Database*) is a Microsoft **COM-based, low-level set of interfaces** for uniform access to data from many different sources - relational (Access, SQL Server, Oracle) and non-relational (Excel, text files, e-mail). It is the core of Microsoft's **Universal Data Access (UDA)** strategy. **Purpose:** one standard way to reach any data store, not only SQL databases (unlike ODBC).

OLE DB component	Role	Example
Data provider	Owns the data and exposes it through OLE DB	Jet 4.0, SQL Server provider
Data consumer	Application/component that uses the data	ADO, VB application
Service component	Adds services such as cursors and query processing	Cursor service

Provider name (used in connection string)	Connects to
`Microsoft.Jet.OLEDB.4.0`	Access (.mdb), Excel, text files
`SQLOLEDB`	Microsoft SQL Server
`MSDAORA`	Oracle
`MSDASQL`	Any **ODBC** data source (OLE DB Provider for ODBC drivers)

OAVS EXAM POINTS

- OLE DB is COM-based and part of UDA.
- It reaches relational **and** non-relational data.
- MSDASQL lets OLE DB/ADO use ODBC drivers.

COMMON MISTAKES / CONFUSING POINTS

- OLE DB is **not** the same as OLE (document linking/embedding).
- OLE DB is not limited to SQL data; VB normally uses it *through ADO*, not directly.

Summary: OLE DB = low-level COM data-access layer with providers for each data source; it is the foundation under ADO.

ii. How OLE DB relates to ADO

Relationship: ADO is the **high-level, easy-to-use interface** built on top of OLE DB. VB code calls ADO objects; ADO calls OLE DB; OLE DB calls the provider, which reaches the data. ADO is therefore an **OLE DB consumer**.

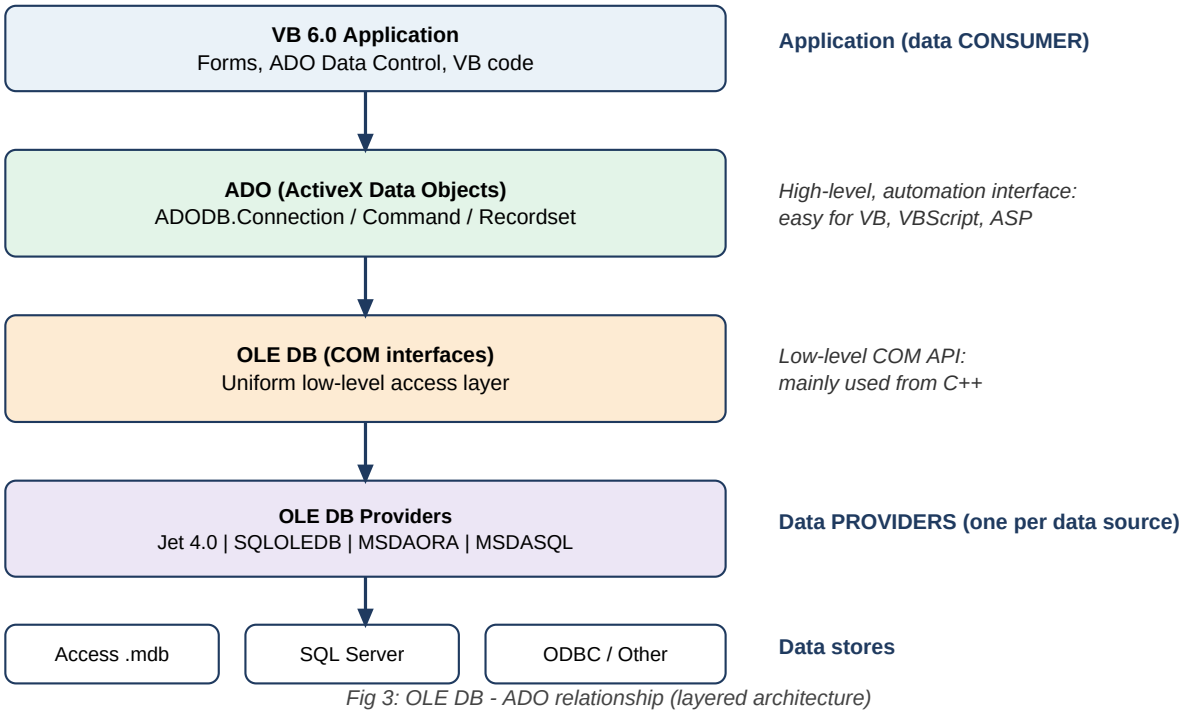

Fig 3: OLE DB - ADO relationship (layered architecture)

ADO object model (data-access architecture)

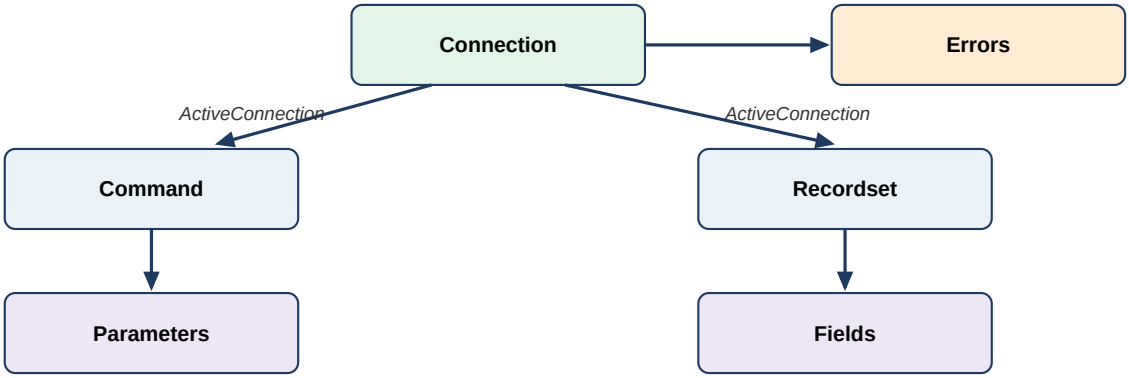

Fig 4: Main ADO objects - Connection, Command, Recordset and their collections

ADO object	Purpose
`Connection`	Establishes and manages the link to the data source; can run SQL via Execute
`Command`	Runs a SQL statement / stored procedure; holds Parameters
`Recordset`	Holds the records returned; used to navigate, add, edit, delete, search
`Field`	One column of the current record (Recordset.Fields collection)
`Parameter / Error`	Command argument / provider error information

To use ADO in code: Project > References > *Microsoft ActiveX Data Objects 2.x Library*; declare objects with the **ADODB** prefix, e.g. `Dim cn As ADODB.Connection`.

	OLE DB	ADO
Level	Low-level system interface	High-level application interface
Technology	COM interfaces (C++ oriented)	ActiveX/automation objects (VB friendly)
Role	Provider layer to the data	Consumer of OLE DB; used by VB code

OAVS EXAM POINTS

- ADO → OLE DB → Provider → Data store.
- ADO objects can be created **independently** (no strict object hierarchy, unlike DAO).
- Reference: Microsoft ActiveX Data Objects Library; prefix ADODB.

COMMON MISTAKES / CONFUSING POINTS

- Do not say ADO accesses the database directly - it always goes through an OLE DB provider.
- Forgetting the reference gives "User-defined type not defined".

Summary: ADO is the VB-friendly face of OLE DB. Main objects: Connection, Command, Recordset (with Fields, Parameters, Errors).

iii. SQL Statements

Definition: SQL (Structured Query Language) is the standard language to retrieve and change data. In ADO the SQL text is passed as a **string** to *RecordSource*, *Recordset.Open* or *Connection.Execute*. (Only the four basic statements are needed here.)

Statement	Purpose	Syntax / example
SELECT	Retrieve records	`SELECT * FROM Student WHERE Marks >= 40 ORDER BY Name`
INSERT	Add a new record	`INSERT INTO Student (RollNo, Name, Marks) VALUES (105, 'Neha', 72)`
UPDATE	Change existing records	`UPDATE Student SET Marks = 80 WHERE RollNo = 104`
DELETE	Remove records	`DELETE FROM Student WHERE RollNo = 105`

Sample data used in the examples

RollNo	Name	Marks	ClassID
101	Ravi	70	C1
102	Asha	58	C2
103	Kiran	85	C1
104	Meena	65	C2

Statement	Output on the sample data
SELECT	4 records, sorted by Name: Asha 58, Kiran 85, Meena 65, Ravi 70
INSERT	1 record added: 105 | Neha | 72
UPDATE	1 record changed: Meena 65 becomes 80
DELETE	1 record removed: 105 Neha

Using SQL from VB (with expected output)

```
cn.Execute "UPDATE Student SET Marks = 80 WHERE RollNo = 104"      'changes 1 record
rs.Open "SELECT Name FROM Student WHERE Name LIKE 'R*'", cn         'Access wildcard *
txtName.Text = "Asha"
rs.Open "SELECT * FROM Student WHERE Name = '" & txtName.Text & "'", cn  'text in quotes
```

OUTPUT

```
Line 1 (UPDATE): 1 record changed  ->  104 | Meena | Marks 65 changed to 80
Line 2 (SELECT): rs holds 1 record  ->  Name = Ravi
Line 3 (SELECT): rs holds 1 record  ->  102 | Asha | 58 | C2
```

Lines 2 and 3 only fill the recordset; nothing appears on the form until the code reads it (e.g. rs!Name).

OAVS EXAM POINTS

- Four basic statements: SELECT, INSERT, UPDATE, DELETE.
- Text values go in single quotes; numbers need none (Access dates use #...#).
- **Connection.Execute** runs action queries.

COMMON MISTAKES / CONFUSING POINTS

- **UPDATE/DELETE without WHERE** affects every record.
- Missing quotes around text, or a name containing an apostrophe (O'Brien), causes a syntax error.

Summary: SQL strings are built in VB and handed to ADO; SELECT reads, INSERT adds, UPDATE modifies, DELETE removes.

iv. ADO Data Control

Definition: The ADO Data Control (**ADODC**) is an ActiveX control that connects to a data source using ADO and supplies a Recordset to bound controls. It shows first / previous / next / last arrow buttons. **Purpose:** data access with little or no code.

Item	Detail
Component name	Microsoft ADO Data Control 6.0 (OLEDB) - file MSADODC.OCX
Default name	`Adodc1`

Using ADO

- Project > Components > tick *Microsoft ADO Data Control 6.0 (OLEDB)*; draw **Adodc1** on the form.
- Set **ConnectionString** and **RecordSource**; set **DataSource = Adodc1** in bound controls and choose **DataField**.
- Run the form: arrow buttons move through records; bound controls update automatically.

Property / Method / Event	Use
ConnectionString	How to connect (provider, database, user)
RecordSource	Table name or SQL statement giving the records
CommandType	Type of RecordSource: 1 adCmdText, 2 adCmdTable, 4 adCmdStoredProc, 8 adCmdUnknown
EOFAction / BOFAction	EOF: 0 MoveLast, 1 StayEOF, 2 AddNew; BOF: 0 MoveFirst, 1 StayBOF
Recordset	The underlying ADO Recordset: `Adodc1.Recordset.AddNew`
Refresh	Re-reads data after RecordSource/ConnectionString change
WillMove / MoveComplete	Events fired before / after moving to another record

Connecting to data source

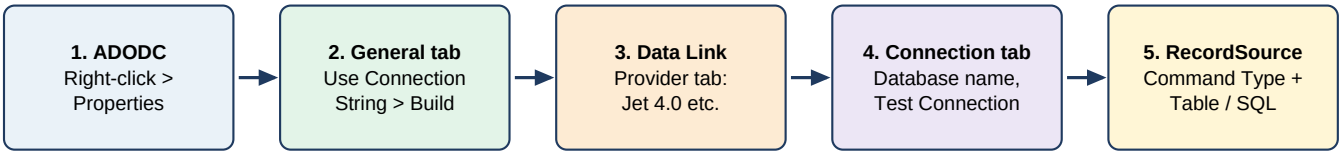

Fig 5: Data-source connection flow for the ADO Data Control

In the General tab there are three choices: **Use Connection String** (Build...), **Use Data Link File** (.udl) and **Use ODBC Data Source Name**. Run-time equivalent:

```
Adodc1.ConnectionString = "Provider=Microsoft.Jet.OLEDB.4.0;Data Source=" & App.Path & "\student.mdb"
Adodc1.CommandType = adCmdText
Adodc1.RecordSource = "SELECT * FROM Student"
Adodc1.Refresh
```

OUTPUT

```
+-Form1 ----------------------+
| RollNo : [ 101   ]          |
| Name   : [ Ravi  ]          |
| Marks  : [ 70    ]          |
|                             |
| [|<] [<]   Adodc1   [>] [>|] |
+-----------------------------+

Adodc1.Recordset.RecordCount = 4   (connected to student.mdb)
```

The text boxes show the first record only when their DataSource = Adodc1 and DataField is set. Clicking [>] moves to 102 | Asha | 58.

Binding controls (DataCombo, DataList, DataGrid)

Data binding links a control to a field of the recordset so that it displays and edits data automatically.

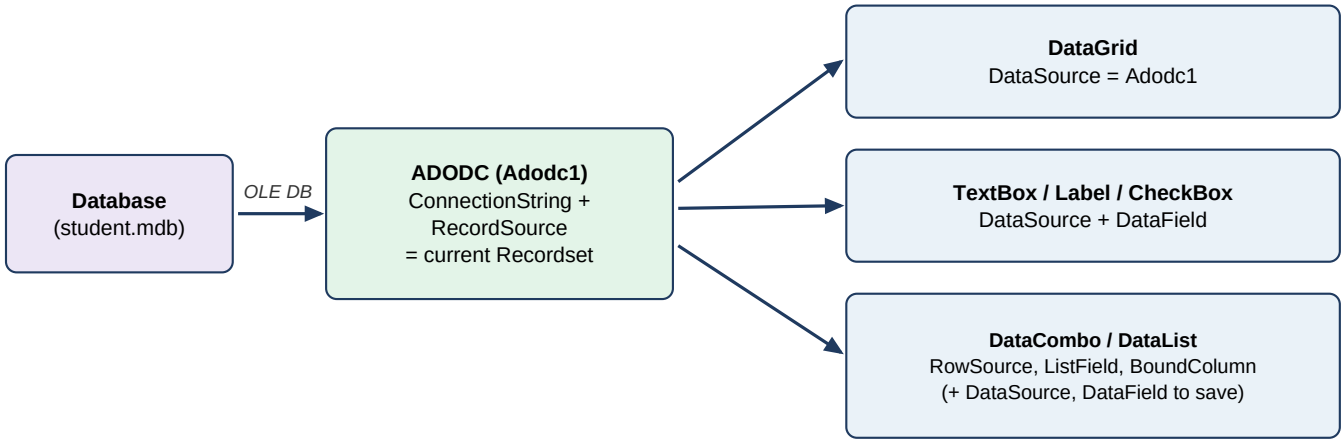

Fig 6: Data-binding flow - Database to ADODC to bound controls

Control	Component (OCX)	Key properties	Purpose
TextBox, Label, CheckBox	Intrinsic	DataSource, DataField	Show/edit one field
DataGrid	Microsoft DataGrid Control 6.0 (OLEDB) - MSDATGRD.OCX	DataSource; AllowAddNew, AllowDelete, AllowUpdate	Editable table view of records
DataList	Microsoft DataList Controls 6.0 (OLEDB) - MSDATLST.OCX	RowSource, ListField, BoundColumn, DataSource, DataField	Always-visible list filled from a recordset
DataCombo	Same file as DataList	Same as DataList + Style	Drop-down combo filled from a recordset

Example (lookup): To store the class chosen from a list into the Student table:

```
DataCombo1.RowSource   = "Adodc2"      'Adodc2 -> Class table (supplies the list)
DataCombo1.ListField   = "ClassName"   'text the user SEES
DataCombo1.BoundColumn = "ClassID"     'value that is STORED
DataCombo1.DataSource  = "Adodc1"      'Adodc1 -> Student table (receives the value)
DataCombo1.DataField   = "ClassID"     'field updated in Student
```

OUTPUT

```
Class table (RowSource)        Before: Student 101 | Ravi | ClassID = C1
ClassID  ClassName
C1       Class X
C2       Class XI

+-DataCombo1 ------------------------------+
| Class XI    v   (user picks the 2nd item) |
+-----------------------------------------+

DataCombo1.Text      = "Class XI"   (what the user sees)
DataCombo1.BoundText = "C2"         (what is stored)
After update: Student 101 | Ravi | ClassID = C2
```

OAVS EXAM POINTS

- ADODC = ADO + data-bound controls, no code needed.
- ADODC file MSADODC.OCX; DataGrid MSDATGRD.OCX; DataList/DataCombo MSDATLST.OCX.
- **RowSource** = where the list comes from; **DataSource** = where the chosen value is saved.
- CommandType: 1 Text, 2 Table, 4 StoredProc.

COMMON MISTAKES / CONFUSING POINTS

- Mixing up RowSource and DataSource in DataCombo/DataList.
- Not setting ListField/BoundColumn - the list shows blank.
- DataGrid shows nothing until DataSource is set (and Retrieve Fields at design time).
- Forgetting **Refresh** after changing RecordSource at run time.

Summary: ADODC connects through a connection string and RecordSource; bound controls use DataSource/DataField; DataCombo/DataList add RowSource/ListField/BoundColumn; DataGrid shows whole recordset.

v. Coding ADO

Definition: Coding ADO means creating and using ADO objects directly in VB code instead of the ADODC. It gives full control over connection, cursor, locking and navigation. **Reference needed:** Microsoft ActiveX Data Objects 2.x Library.

Creating record set

```
Dim cn As ADODB.Connection
Dim rs As ADODB.Recordset
Set cn = New ADODB.Connection
cn.Open "Provider=Microsoft.Jet.OLEDB.4.0;Data Source=" & App.Path & "\student.mdb"
Set rs = New ADODB.Recordset
rs.Open "SELECT * FROM Student", cn, adOpenKeyset, adLockOptimistic, adCmdText
Do While Not rs.EOF
    List1.AddItem rs!Name
    rs.MoveNext
Loop
rs.Close: cn.Close: Set rs = Nothing: Set cn = Nothing
```

OUTPUT

```
+-List1 --+
| Ravi    |
| Asha    |
| Kiran   |
| Meena   |
+---------+

rs.RecordCount = 4 (keyset cursor); the loop stops when rs.EOF = True.
```

Syntax: `rs.Open Source, ActiveConnection, CursorType, LockType, Options.`

Argument	Common values (numeric value)	Default
CursorType	adOpenForwardOnly (0), adOpenKeyset (1), adOpenDynamic (2), adOpenStatic (3)	adOpenForwardOnly
LockType	adLockReadOnly (1), adLockPessimistic (2), adLockOptimistic (3), adLockBatchOptimistic (4)	adLockReadOnly
Options	adCmdText (1), adCmdTable (2), adCmdStoredProc (4)	-

Recordset member	Meaning
EOF / BOF	True when positioned after the last / before the first record (empty recordset: both True)
RecordCount	Number of records; **-1** for a forward-only cursor
MoveFirst, MoveLast, MoveNext, MovePrevious	Navigation methods
Fields("Name").Value or rs!Name	Read / write a field of the current record
State	0 adStateClosed, 1 adStateOpen
Close	Closes the recordset (also close Connection)

Adding records in record set

```
rs.AddNew                        'blank new record
rs!RollNo = Val(txtRoll.Text)
rs!Name   = txtName.Text
rs!Marks  = Val(txtMarks.Text)
rs.Update                        'saves the new record
```

OUTPUT

```
Input: txtRoll = 105, txtName = Neha, txtMarks = 72  ->  after rs.Update the table is:

RollNo  Name    Marks  ClassID
101     Ravi    70     C1
102     Asha    58     C2
103     Kiran   85     C1
104     Meena   65     C2
105     Neha    72     (empty)   <-- new record

rs.RecordCount changes from 4 to 5.
```

Use `rs.CancelUpdate` to discard a new record. A read-only lock gives the error "Current Recordset does not support updating". Alternative: `cn.Execute "INSERT INTO ..."`.

Modify, Delete, Search

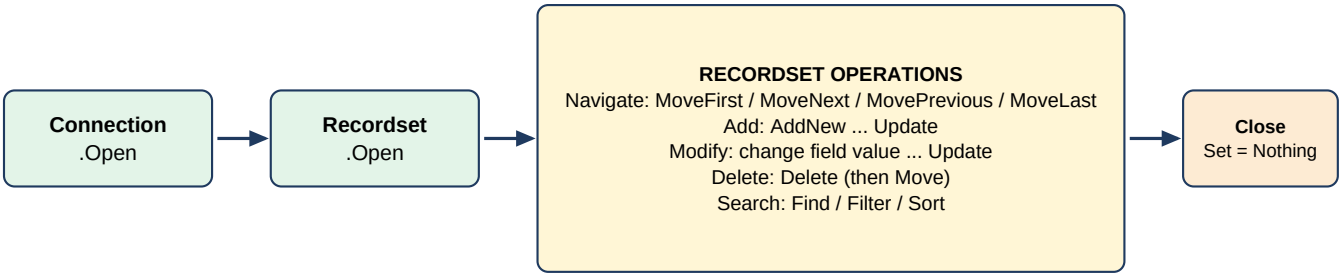

Fig 7: Recordset operations - open, work on records, close

```
'--- Search (Find) then MODIFY ---
rs.MoveFirst
rs.Find "RollNo = 101"            'criteria: one field, operator, value
If Not rs.EOF Then
    rs!Marks = 85                 'modify - there is NO Edit method in ADO
    rs.Update
Else
    MsgBox "Record not found"
End If
'--- DELETE ---
rs.Delete                         'deletes current record
rs.MoveNext                       'move off the deleted record
If rs.EOF Then rs.MoveLast
'--- FILTER / SORT ---
rs.Filter = "Marks > 60"          'shows only matching records;  rs.Filter = adFilterNone clears
rs.Sort = "Name ASC"              'needs CursorLocation = adUseClient
```

OUTPUT

```
rs.Find "RollNo = 101"      -> found; current record = 101 | Ravi | 70
rs!Marks = 85 : rs.Update  -> 101 | Ravi | 85   (saved)
   (if not found, e.g. RollNo = 999: rs.EOF = True and MsgBox shows "Record not found")
rs.Delete                   -> record 101 Ravi is removed
rs.MoveNext                 -> pointer now on 102 | Asha | 58
rs.Filter = "Marks > 60"  -> visible: 103 Kiran 85, 104 Meena 65
rs.Sort = "Name ASC"      -> order: Kiran, Meena
```

Delete removed Ravi because he was the current record at that moment.

Task	Method	Note
Add	AddNew ... Update	Fields are blank until assigned
Modify	Change value ... Update	Moving to another record also saves
Delete	Delete, then Move	Current record becomes invalid
Search	Find "field op value"	Single field; check EOF; MoveFirst before re-searching
Search (subset)	Filter	Condition string; may use AND/OR

OAVS EXAM POINTS

- ADO: **AddNew** → **Update**; modify: assign + **Update** (no Edit method, unlike DAO).
- Default CursorType = ForwardOnly, default LockType = ReadOnly; use Keyset/Optimistic to edit.
- Find not successful → **EOF = True** (searching forward).
- RecordCount = -1 for forward-only cursor.

COMMON MISTAKES / CONFUSING POINTS

- Writing `rs.Edit` (DAO habit) - causes error in ADO.
- Forgetting **Update** after AddNew/changes.
- Using the record right after Delete without moving ("current record has been deleted").
- Find with two conditions joined by AND (not allowed; use Filter).
- Not testing EOF after Find.

Summary: Coding ADO: open Connection → open Recordset (cursor + lock) → navigate / AddNew+Update / edit+Update / Delete / Find or Filter → close.

vi. Data Environment

Definition: The Data Environment is a VB 6.0 **design-time designer** (Project > Add Data Environment) in which you create **Connection** and **Command** objects visually. They are available to the whole project as a reusable *DataEnvironment* object (saved as a .dsr file). **Purpose:** define data access once and use it from many forms, including drag-and-drop creation of bound controls.

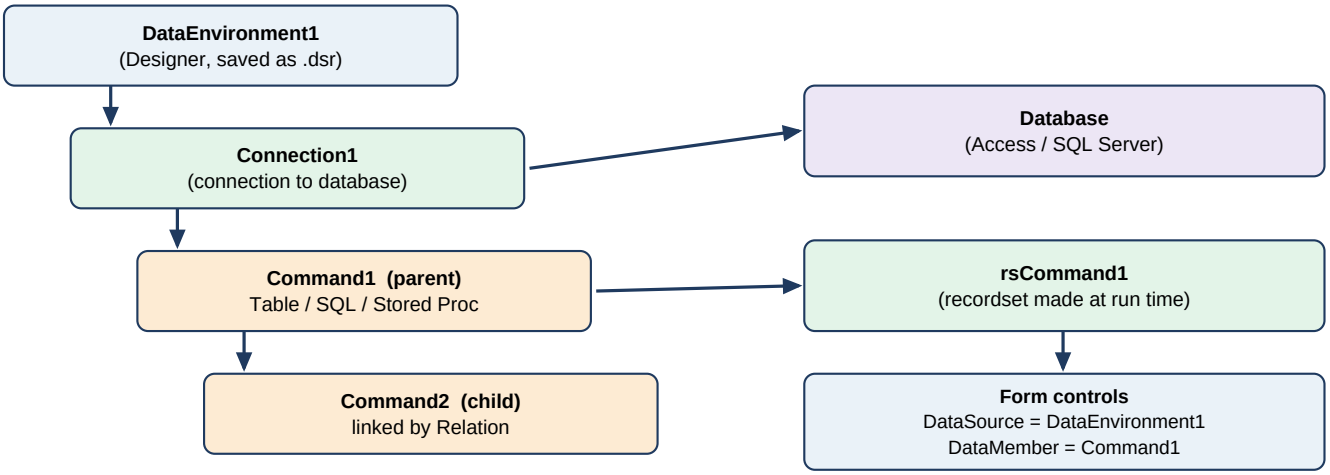

Fig 8: Data Environment structure

Element	Purpose
DataEnvironment1	The designer object that holds everything
Connection1	Provider and database details (like ADO Connection)
Command1	Table, view, stored procedure or SQL statement (like ADO Command)
Child command	Command nested under a parent via a Relation (hierarchical data)
rsCommand1	Recordset created at run time for Command1 (name = rs + command name)

Steps: (1) Add Data Environment; (2) Connection1 > Properties > choose provider and database; (3) right-click Connection1 > *Add Command* and set Database Object / SQL; (4) drag Command1 onto a form to create bound controls, or bind manually:

```
Set DataGrid1.DataSource = DataEnvironment1
DataGrid1.DataMember = "Command1"
DataEnvironment1.rsCommand1.MoveNext
Text1.Text = DataEnvironment1.rsCommand1.Fields("Name")
```

OUTPUT

```
DataGrid1 (DataMember = "Command1", Command1 = SELECT * FROM Student)

RollNo  Name    Marks  ClassID
101     Ravi    70     C1
102     Asha    58     C2
103     Kiran   85     C1
104     Meena   65     C2

After rsCommand1.MoveNext (from the first record):  Text1.Text = Asha
```

	ADO Data Control	Data Environment	Coding ADO
Form	Control on a form	Designer for whole project	Code
Reuse	One per form	Shared by all forms	Written wherever needed
Code needed	Little	Little	Most

OAVS EXAM POINTS

- Designer object: Connection + Command; recordsets are named **rs<CommandName>**.
- Binding needs **DataSource = DataEnvironment1** and **DataMember = command name**.
- Child commands give hierarchical (parent-child) data.

COMMON MISTAKES / CONFUSING POINTS

- Forgetting DataMember.
- Using rsCommand1 as if it were the command name (it is the recordset).
- Confusing Data Environment (project designer) with the ADODC (control).

Summary: Data Environment = reusable Connection and Command objects defined in a designer, exposing recordsets (rsCommand) to bound controls.

vii. Introduction to Crystal Report (7.0 or above)

Definition: Crystal Reports is a report-writing tool (from Seagate, later Crystal Decisions/Business Objects) that works with VB to design formatted, printable reports from database data. **Purpose:** produce grouped, totalled, print-ready reports (.rpt files) that VB can preview or print.

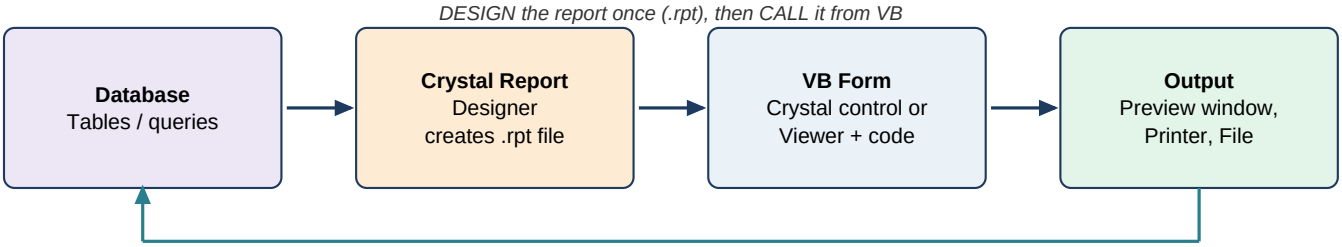

Fig 9: Crystal Report data flow

Report section	Printed
Report Header / Report Footer	Once at start / end (title, grand total)
Page Header / Page Footer	At top / bottom of every page (column titles, page number)
Group Header / Group Footer	Before / after each group (group name, subtotal)
Details	Once for every record

Steps: create a Standard Report → choose database and tables → select fields → (optional) group, sort and summarize → preview → save as **.rpt** → call it from VB. Exact control names depend on the Crystal version:

```
'Crystal Report control (OCX) method
CrystalReport1.ReportFileName = App.Path & "\student.rpt"
CrystalReport1.SelectionFormula = "{Student.Marks} > 60"
CrystalReport1.Destination = crptToWindow      '0 window, 1 printer, 2 file
CrystalReport1.Action = 1                      'run the report

'Report Designer Component (7.0 and above) with Viewer
Dim crApp As New CRAXDRT.Application
Dim crRpt As CRAXDRT.Report
Set crRpt = crApp.OpenReport(App.Path & "\student.rpt")
CRViewer1.ReportSource = crRpt
CRViewer1.ViewReport
```

OUTPUT

```
+-Crystal Report Preview -------------------+
| Student Marks Report         (Marks > 60) |
|                                           |
| RollNo   Name     Marks                   |
| 101      Ravi     70                      |
| 103      Kiran    85                      |
| 104      Meena    65                      |
|                                           |
| Total records: 3          Page 1 of 1     |
+-------------------------------------------+
```

Asha (58) is left out by the selection formula. The second (viewer) example has no selection formula, so it would show all four students.

OAVS EXAM POINTS

- Report file extension: **.rpt**.
- Sections: Report Header, Page Header, Group Header, Details, Group Footer, Page Footer, Report Footer.
- Crystal field syntax uses braces: **{Table.Field}**.
- Destination: window, printer, file.

COMMON MISTAKES / CONFUSING POINTS

- Report shows old data if "Save Data with Report" is on - refresh/discard saved data.
- Wrong .rpt path gives a file-not-found error.
- Report design (Crystal) vs report display (VB code) are separate steps.

Summary: Crystal Reports designs a .rpt file once; VB loads it through the Crystal control or viewer and shows it in a window, printer or file.

Quick Revision

Key definitions

Term	One-line definition
Data access interface	API that lets VB connect to and use a database (DAO, RDO, ADO)
Relational database	Data stored in related tables linked by primary/foreign keys
OLE DB	Low-level COM interface giving uniform access to relational and non-relational data (UDA)
ADO	High-level object interface built over OLE DB; used from VB
SQL	Language: SELECT, INSERT, UPDATE, DELETE
ADODC	ActiveX control that connects through ADO and feeds bound controls
Data binding	Linking a control to a recordset field (DataSource, DataField)
Data Environment	Designer holding reusable Connection and Command objects
Crystal Report	Report tool creating .rpt files shown from VB

Important components

Item	File / name
ADO Data Control	Microsoft ADO Data Control 6.0 (OLEDB) - MSADODC.OCX
DataGrid	Microsoft DataGrid Control 6.0 (OLEDB) - MSDATGRD.OCX
DataList / DataCombo	Microsoft DataList Controls 6.0 (OLEDB) - MSDATLST.OCX
ADO library	Microsoft ActiveX Data Objects 2.x Library (ADODB prefix)
Jet provider	`Microsoft.Jet.OLEDB.4.0`
Data Environment recordset	`rs` + command name, e.g. `rsCommand1`

ADO / OLE DB relationships

Interface	Based on / works through
DAO	Jet engine (Access)
RDO	ODBC (client/server)
ADO	OLE DB
VB → ADO → OLE DB → Provider → Data store	MSDASQL provider reaches ODBC sources

Recordset operations and constants

Operation	Code
Open	`rs.Open src, cn, CursorType, LockType, Options`
Navigate	`MoveFirst, MoveNext, MovePrevious, MoveLast; EOF / BOF`
Add	`AddNew ... Update` (cancel: `CancelUpdate`)
Modify	Assign new value ... `Update` (no Edit)
Delete	`Delete` then `MoveNext/MoveLast`
Search	`Find "field = value"`, `Filter`, `Sort`
Close	`rs.Close, cn.Close`

CursorType	LockType	CommandType
0 ForwardOnly, 1 Keyset, 2 Dynamic, 3 Static	1 ReadOnly, 2 Pessimistic, 3 Optimistic, 4 BatchOptimistic	1 Text, 2 Table, 4 StoredProc, 8 Unknown

Likely exam points

- ADO is the VB 6.0 interface built on OLE DB; DAO uses Jet; RDO uses ODBC.
- OLE DB = Object Linking and Embedding Database (COM, UDA); not limited to SQL data.
- ADO Data Control file is MSADODC.OCX; bound controls use **DataSource** and **DataField**.
- DataCombo/DataList: **RowSource** (list source), **ListField** (displayed), **BoundColumn** (stored value).
- ADODC properties: ConnectionString, RecordSource, CommandType (1 Text, 2 Table, 4 StoredProc).
- Default ADO recordset: forward-only, read-only; use Keyset + Optimistic to add/edit.
- Adding/modifying needs **Update**; there is no Edit method in ADO.
- Find failure leaves **EOF = True**; RecordCount is -1 for forward-only cursors.

- Data Environment: Connection + Command objects; recordset name rsCommandName; binding uses DataMember.
- Crystal Report files are .rpt; sections include Report/Page/Group Header, Details, Footers.